

Rapport hebdomadaire : Fine-tuning pour segmentation prosodique

TROMBATI Mohamed

Semaine du 22 au 26 juin

1 Introduction

Durant cette semaine, j'ai travaillé sur une nouvelle formulation du problème de segmentation prosodique sous la forme d'une prédiction temporelle continue à partir d'embeddings extraits de modèles de type Transformer pré-entraînés (Pantagruel-B-1K, Pantagruel-B-14K et Pantagruel-L-14K).

L'objectif de ce travail était de transformer une séquence acoustique en un signal de probabilité indiquant la présence de frontières prosodiques. Contrairement à une approche classique de classification binaire frame-par-frame, j'ai choisi de formuler le problème comme une tâche de régression sur un signal continu.

Cette formulation a été motivée par la nature même des frontières prosodiques. En effet, celles-ci ne correspondent pas à des événements ponctuels parfaitement définis dans le temps. Une frontière peut apparaître progressivement et sa position exacte peut varier selon plusieurs facteurs tels que le locuteur, le débit de parole ou encore les conditions acoustiques d'enregistrement.

Une classification binaire stricte impose donc une séparation artificielle entre une frame considérée comme une frontière et une frame considérée comme n'en étant pas une. Cette hypothèse peut être problématique lorsque les annotations possèdent une certaine incertitude, notamment lorsque les frontières sont obtenues automatiquement à partir de pauses détectées dans le signal audio.

L'approche que j'ai adoptée consiste donc à apprendre une représentation continue de la probabilité de présence d'une frontière autour des positions annotées. Cette stratégie permet au modèle d'être plus robuste face au bruit des annotations et d'apprendre une notion de proximité plutôt qu'une décision binaire stricte.

2 Formulation du problème et choix méthodologiques

2.1 Nature du problème

Le problème de segmentation est ici reformulé comme une tâche de régression sur un signal temporel.

Au lieu de demander au modèle de prédire une étiquette binaire indiquant uniquement la présence ou l'absence d'une frontière à chaque instant, j'ai choisi de lui faire apprendre un score continu représentant la probabilité qu'une frontière soit présente à chaque position temporelle.

Cette formulation présente plusieurs avantages. Premièrement, elle permet d'intégrer directement l'incertitude présente dans les annotations. Dans les jeux de données de parole, les frontières prosodiques sont souvent générées automatiquement à partir de caractéristiques acoustiques comme les pauses ou les variations d'énergie. Ces méthodes introduisent nécessairement une approximation puisque la frontière linguistique réelle n'est pas toujours parfaitement alignée avec un événement acoustique précis.

Deuxièmement, cette approche permet d'obtenir un apprentissage plus stable. Une cible binaire impose au modèle une transition brutale entre deux états différents, alors qu'une cible continue fournit une information plus progressive sur la proximité d'une frontière.

2.2 Construction des labels continus

Afin de représenter cette incertitude temporelle, j’ai construit les labels sous forme de fonctions gaussiennes centrées sur les positions des frontières réelles.

Chaque frontière ne correspond donc pas uniquement à une frame unique, mais influence également les frames voisines avec une intensité décroissante.

Mathématiquement, pour une frontière située en position t_i , le score associé est défini par :

$$y_t = \max_i \exp \left(-\frac{(t - t_i)^2}{2\sigma^2} \right)$$

où σ contrôle la largeur de la zone d’influence autour de la frontière.

Cette représentation permet au modèle d’apprendre une notion de proximité temporelle. Une prédiction légèrement décalée par rapport à la frontière réelle n’est donc pas considérée comme totalement incorrecte pendant l’apprentissage.

Ce choix est particulièrement adapté au contexte de la segmentation prosodique, car une différence de quelques frames entre une prédiction et une annotation humaine ou automatique peut être difficilement interprétable.

2.3 Justification du choix de la fonction de perte

Plusieurs fonctions de perte peuvent être utilisées pour cette tâche, notamment la `MSELoss` ou la `BCEWithLogitsLoss`.

Dans mes expérimentations, j’ai choisi une supervision continue basée principalement sur une `MSELoss`. Ce choix permet au modèle de reproduire progressivement la forme du signal cible au lieu de rechercher uniquement une séparation entre deux classes.

L’utilisation d’un signal continu agit également comme une forme de régularisation. Elle limite le risque que le modèle mémorise précisément des positions de frontières potentiellement bruitées et favorise une meilleure capacité de généralisation.

Cette formulation est donc cohérente avec la nature du problème étudié, où la frontière exacte est moins importante que la capacité du modèle à identifier correctement les zones de rupture prosodique.

3 Architecture du système

3.1 Pipeline global

Le système développé durant cette semaine repose sur plusieurs étapes successives.

Dans un premier temps, les signaux audio sont encodés grâce aux modèles pré-entraînés de la famille `Pantagruel`. Ces modèles permettent d’obtenir des représentations vectorielles riches contenant des informations acoustiques et linguistiques.

Les embeddings extraits sont ensuite sauvegardés dans un cache afin de séparer la phase coûteuse d’extraction des représentations de la phase d’apprentissage. Cette organisation permet de réduire considérablement le temps nécessaire aux expérimentations et facilite la comparaison entre différents modèles.

À partir de ces représentations, une tête de segmentation légère est entraînée afin de prédire un score de frontière pour chaque frame.

3.2 Module dataset

Le module `dataset.py` constitue une étape importante de la préparation des données.

J’ai implémenté une gestion du padding dynamique permettant de traiter des séquences audio de longueurs différentes dans un même batch. Cette étape est indispensable puisque chaque segment audio possède une durée variable.

J’ai également ajouté un masque permettant d’ignorer les positions artificielles ajoutées lors du padding. Ainsi, ces positions n’influencent ni l’apprentissage ni le calcul de la fonction de perte.

Le dataset associe également à chaque séquence son label continu correspondant afin de conserver la cohérence temporelle entre les embeddings et les frontières attendues.

3.3 Segmentation head

La tête de segmentation repose sur une architecture légère permettant d’exploiter les informations temporelles présentes dans les embeddings.

J’ai utilisé une architecture capable de prendre en compte les dépendances locales et semi-globales entre les différentes frames audio.

Une attention particulière a été portée à la gestion du padding grâce aux mécanismes de masquage et à l’utilisation de séquences compressées avec `pack_padded_sequence`. Cette étape permet d’éviter que les zones artificielles introduites par le padding perturbent l’apprentissage du modèle.

4 Apprentissage et stratégie d’entraînement

4.1 Optimisation

L’entraînement repose sur une minimisation progressive de l’écart entre le signal prédit par le modèle et le signal cible généré à partir des frontières prosodiques.

J’ai mis en place une stratégie d’apprentissage avec une surveillance de la performance sur un ensemble de validation. Afin d’éviter un phénomène de sur-apprentissage, j’ai utilisé un mécanisme d’early stopping basé sur l’évolution de la validation loss.

Cette stratégie permet de conserver uniquement le modèle ayant obtenu les meilleures performances durant l’entraînement. Elle est particulièrement importante dans ce contexte car les annotations utilisées pour la segmentation contiennent une certaine quantité de bruit, ce qui peut entraîner une dégradation des performances lorsque le modèle continue à apprendre au-delà d’un certain nombre d’époques.

J’ai observé une convergence rapide durant les premières époques d’entraînement, suivie d’un plateau relativement stable. Cette évolution indique que le modèle apprend rapidement les informations principales contenues dans les embeddings pré-entraînés avant d’atteindre une limite liée davantage aux données qu’à l’architecture utilisée.

4.2 Stabilité de l’apprentissage

Un point important observé durant les entraînements concerne la stabilité du processus d’apprentissage.

Contrairement à une tâche classique de classification binaire, la formulation continue permet au modèle d’apprendre progressivement la structure temporelle des frontières. Le modèle commence par identifier les zones présentant une forte probabilité de rupture prosodique puis affine progressivement la position des pics prédits.

J’ai constaté que les différentes architectures testées présentent un comportement similaire, avec une diminution rapide de la loss durant les premières époques puis une stabilisation autour d’une valeur minimale.

5 Résultats expérimentaux

J’ai évalué trois variantes du modèle Pantagruel dans le cadre du fine-tuning pour la tâche de détection de frontières prosodiques :

- Pantagruel-B-1K : modèle de taille réduite bénéficiant d’un corpus de pré-entraînement plus limité.
- Pantagruel-B-14K : modèle de taille base entraîné sur un volume plus important de données.
- Pantagruel-L-14K : modèle large disposant d’une capacité de représentation supérieure.

L’objectif de cette comparaison était d’étudier l’influence de la taille du modèle et de la quantité de données de pré-entraînement sur une tâche de segmentation temporelle fine.

5.1 Résultats Pantagruel-B-1K

Le modèle Pantagruel-B-1K montre une convergence rapide durant les premières époques.

La forte diminution de la loss entre la première et la deuxième époque indique que la tête de segmentation apprend rapidement à exploiter les informations contenues dans les embeddings extraits du modèle pré-entraîné.

Après environ huit époques, j’ai observé une stabilisation des performances autour d’un F1-score de 0.30. Le meilleur résultat obtenu pendant l’entraînement correspond à un F1-score de 0.3042 avec une validation loss de 0.0186.

L’early stopping a été déclenché à l’époque 13 afin d’éviter une dégradation liée au sur-apprentissage.

Epoch	Train Loss	Validation Loss	Validation F1
1	0.0834	0.0247	0.2478
2	0.0234	0.0209	0.2786
3	0.0212	0.0202	0.2878
4	0.0201	0.0200	0.2967
5	0.0194	0.0190	0.2946
6	0.0188	0.0189	0.2902
7	0.0183	0.0189	0.2951
8	0.0178	0.0186	0.3000
12	0.0162	0.0198	0.3042

TABLE 1 – Evolution de l’entraînement du modèle Pantagruel-B-1K

5.2 Résultats Pantagruel-B-14K

Le modèle Pantagruel-B-14K obtient les meilleures performances globales parmi les modèles évalués.

J’ai obtenu un meilleur F1-score de 0.3090, ce qui représente le meilleur résultat observé pendant cette phase d’entraînement.

L’amélioration par rapport au modèle B-1K reste cependant limitée. Cela indique que l’augmentation de la quantité de données de pré-entraînement apporte un gain, mais que ce gain reste contraint par la difficulté intrinsèque de la tâche et la qualité des annotations.

Epoch	Train Loss	Validation Loss	Validation F1
1	0.0816	0.0246	0.2481
2	0.0229	0.0207	0.2830
3	0.0207	0.0197	0.2928
4	0.0198	0.0194	0.3028
5	0.0191	0.0191	0.2977
6	0.0185	0.0188	0.3059
7	0.0180	0.0188	0.2949
8	0.0176	0.0190	0.2994
11	0.0163	0.0188	0.3090

TABLE 2 – Evolution de l’entraînement du modèle Pantagruel-B-14K

5.3 Résultats Pantagruel-L-14K

Le modèle Pantagruel-L-14K présente une convergence particulièrement rapide.

Dès les premières époques, j’ai obtenu des performances proches des meilleurs résultats observés avec les autres architectures.

Cependant, malgré sa capacité de représentation supérieure, ce modèle ne dépasse pas Pantagruel-B-14K. Cette observation suggère que la limite principale ne semble pas uniquement liée à la capacité du modèle mais également au bruit présent dans les annotations.

Epoch	Train Loss	Validation Loss	Validation F1
1	0.0838	0.0236	0.2868
2	0.0220	0.0211	0.2567
3	0.0195	0.0197	0.2997
4	0.0182	0.0191	0.3023
5	0.0172	0.0192	0.3027
6	0.0162	0.0192	0.3044
7	0.0153	0.0201	0.3035
8	0.0147	0.0206	0.2977

TABLE 3 – Evolution de l’entraînement du modèle Pantagruel-L-14K

5.4 Comparaison finale des modèles

Le tableau suivant présente une synthèse des meilleures performances obtenues durant les entraînements.

Modèle	Meilleur Epoch	Best Validation Loss	Best Validation F1	Early stopping
Pantagruel-B-1K	8	0.0186	0.3042	Epoch 13
Pantagruel-B-14K	11	0.0188	0.3090	Epoch 12
Pantagruel-L-14K	6	0.0191	0.3044	Epoch 9

TABLE 4 – Comparaison globale des modèles fine-tunés

6 Analyse des résultats

Les résultats obtenus montrent une forte homogénéité entre les trois architectures testées.

J’ai constaté que les performances finales se situent dans une plage comprise entre 0.30 et 0.31 en F1-score. La différence entre le meilleur modèle et les autres modèles reste donc relativement faible.

Cette observation m’a permis de conclure que l’augmentation de la taille du modèle n’est pas nécessairement le facteur principal permettant d’améliorer les performances sur cette tâche.

Le modèle Pantagruel-B-14K fournit le meilleur compromis entre capacité de représentation et généralisation, tandis que Pantagruel-L-14K, malgré un nombre plus important de paramètres, ne permet pas d’obtenir une amélioration significative.

Le modèle Pantagruel-B-1K reste également compétitif, ce qui montre que les embeddings extraits contiennent déjà une quantité importante d’informations utiles pour la segmentation prosodique.

7 Discussion

7.1 Limites fondamentales

La principale limitation que j’ai identifiée concerne la qualité des annotations utilisées.

Les frontières prosodiques étant obtenues automatiquement à partir de pauses acoustiques, elles représentent seulement une approximation de la structure linguistique réelle.

Cette approximation introduit un bruit qui limite le niveau maximal de performance pouvant être atteint. Même un modèle disposant d'une capacité de représentation importante reste limité par l'incertitude présente dans les données d'apprentissage.

7.2 Impact du choix continu

L'approche continue que j'ai adoptée permet de réduire partiellement cette difficulté.

Au lieu d'imposer au modèle de prédire une position exacte, je lui demande d'apprendre une distribution autour des frontières attendues.

Cette formulation est davantage adaptée à la nature du problème et explique probablement la stabilité observée durant les différentes phases d'entraînement.

8 Conclusion

Durant cette semaine, j'ai mis en place et évalué une approche de fine-tuning basée sur des embeddings pré-entraînés pour la détection de frontières prosodiques.

Les résultats obtenus montrent qu'une supervision continue combinée à une architecture légère permet d'obtenir une segmentation robuste sans nécessiter une architecture complexe.

Les trois modèles testés présentent des performances similaires autour de 0.30 F1-score, avec un léger avantage pour Pantagruel-B-14K.

Ces premières expérimentations constituent une base solide avant l'évaluation finale sur le jeu de test.

8.1 Travaux prévus pour la semaine prochaine

La semaine prochaine, je prévois de réaliser l'évaluation finale des différents modèles sur le jeu de test afin de comparer leurs performances en conditions réelles.

Je compte également intégrer d'autres modèles de la famille LeBenchmark, comme cela avait été réalisé précédemment dans l'approche zero-shot, afin d'étudier leur comportement dans un contexte de fine-tuning supervisé.